Module SY5 – Systèmes d'Exploitation

Dominique Poulalhon
dominique.poulalhon@irif.fr

Université Paris Cité
L3 Informatique & DL Info-Bio, Info-Jap, Math-Info
Année universitaire 2024-2025

Duplication de descripteur et redirection

il est possible de définir un *synonyme* `fd2` d'un descripteur `fd1`, c'est-à-dire de faire en sorte que `fd2` pointe sur la même ligne de la table des ouvertures de fichiers que `fd1` :

```
int dup2(int fd1, int fd2);
int dup(int fd1); /* variante dans laquelle fd2 est choisi par le
  système parmi les descripteurs non alloués */
```

- `fd1` doit être un descripteur valide
- `fd2` est un descripteur quelconque, alloué ou non

Duplication de descripteur et redirection

il est possible de définir un *synonyme* `fd2` d'un descripteur `fd1`, c'est-à-dire de faire en sorte que `fd2` pointe sur la même ligne de la table des ouvertures de fichiers que `fd1` :

```
int dup2(int fd1, int fd2);
int dup(int fd1); /* variante dans laquelle fd2 est choisi par le
  système parmi les descripteurs non alloués */
```

- `fd1` doit être un descripteur valide
- `fd2` est un descripteur quelconque, alloué ou non

si `fd2` est déjà alloué, le système le ferme (comme un `close(fd2)`) avant la duplication

Duplication de descripteur et redirection

il est possible de définir un *synonyme* `fd2` d'un descripteur `fd1`, c'est-à-dire de faire en sorte que `fd2` pointe sur la même ligne de la table des ouvertures de fichiers que `fd1` :

```
int dup2(int fd1, int fd2);
int dup(int fd1); /* variante dans laquelle fd2 est choisi par le
  système parmi les descripteurs non alloués */
```

- `fd1` doit être un descripteur valide
- `fd2` est un descripteur quelconque, alloué ou non

si `fd2` est déjà alloué, le système le ferme (comme un `close(fd2)`) avant la duplication

la valeur de retour est `fd2`, ou -1 en cas d'échec

Duplication de descripteur et redirection

il est possible de définir un *synonyme* `fd2` d'un descripteur `fd1`, c'est-à-dire de faire en sorte que `fd2` pointe sur la même ligne de la table des ouvertures de fichiers que `fd1` :

```
int dup2(int fd1, int fd2);
int dup(int fd1); /* variante dans laquelle fd2 est choisi par le
  système parmi les descripteurs non alloués */
```

- `fd1` doit être un descripteur valide
- `fd2` est un descripteur quelconque, alloué ou non

si `fd2` est déjà alloué, le système le ferme (comme un `close(fd2)`) avant la duplication

la valeur de retour est `fd2`, ou -1 en cas d'échec

en particulier, `fd1` et `fd2` partagent alors la même position courante (*offset*)

Duplication de descripteur et redirection

il est possible de définir un *synonyme* `fd2` d'un descripteur `fd1`, c'est-à-dire de faire en sorte que `fd2` pointe sur la même ligne de la table des ouvertures de fichiers que `fd1` :

```
int dup2(int fd1, int fd2);
int dup(int fd1); /* variante dans laquelle fd2 est choisi par le
  système parmi les descripteurs non alloués */
```

utilité : principalement pour réaliser les ***redirections*** des flots standard

Atomicité

problématique omniprésente : lorsque deux actions sont nécessaires, comment s'assurer que rien ne s'est produit entre les deux qui annihile les effets de la 1re ? ou que la 2e sera toujours possible une fois la 1re effectuée ?

Atomicité

problématique omniprésente : lorsque deux actions sont nécessaires, comment s'assurer que rien ne s'est produit entre les deux qui annihile les effets de la 1re ? ou que la 2e sera toujours possible une fois la 1re effectuée ?

certains appels système offrent cette garantie d'atomicité, *i.e.* de non interruption

Atomicité

problématique omniprésente : lorsque deux actions sont nécessaires, comment s'assurer que rien ne s'est produit entre les deux qui annihile les effets de la 1re ? ou que la 2e sera toujours possible une fois la 1re effectuée ?

certains appels système offrent cette garantie d'atomicité, *i.e.* de non interruption

ouverture en `O_CREAT | O_EXCL`
vs test ***puis*** création (en deux appels système)

Atomicité

problématique omniprésente : lorsque deux actions sont nécessaires, comment s'assurer que rien ne s'est produit entre les deux qui annihile les effets de la 1re ? ou que la 2e sera toujours possible une fois la 1re effectuée ?

certains appels système offrent cette garantie d'atomicité, *i.e.* de non interruption

ouverture en `O_CREAT | O_EXCL`
vs test ***puis*** création (en deux appels système)

ouverture en `O_APPEND` $\Longrightarrow$ déplacement en fin de fichier avant chaque écriture de manière atomique
vs `lseek` ***puis*** `write`

Atomicité

problématique omniprésente : lorsque deux actions sont nécessaires, comment s'assurer que rien ne s'est produit entre les deux qui annihile les effets de la 1re ? ou que la 2e sera toujours possible une fois la 1re effectuée ?

certains appels système offrent cette garantie d'atomicité, *i.e.* de non interruption

ouverture en `O_CREAT | O_EXCL`
vs test ***puis*** création (en deux appels système)

ouverture en `O_APPEND` $\Longrightarrow$ déplacement en fin de fichier avant chaque écriture de manière atomique
vs `lseek` ***puis*** `write`

duplication avec `dup2`
vs `close` ***puis*** `dup`

Organisation du système de fichiers

Où et comment stocker les fichiers ?

un fichier, c'est...

Où et comment stocker les fichiers ?

un fichier, c'est...

- du contenu

Où et comment stocker les fichiers ?

un fichier, c'est...

- du contenu
- des attributs ou méta-données (type, taille, permissions, dates...)

Où et comment stocker les fichiers ?

un fichier, c'est...

- du contenu
- des attributs ou méta-données (type, taille, permissions, dates...)

plusieurs solutions :

Où et comment stocker les fichiers ?

un fichier, c'est...

- du contenu
- des attributs ou méta-données (type, taille, permissions, dates...)

plusieurs solutions :

- stockage contigu (CD-ROM)

Où et comment stocker les fichiers ?

un fichier, c'est...

- du contenu
- des attributs ou méta-données (type, taille, permissions, dates...)

plusieurs solutions :

- stockage contigu (CD-ROM)
- liste chaînée de blocs

Où et comment stocker les fichiers ?

un fichier, c'est...

- du contenu
- des attributs ou méta-données (type, taille, permissions, dates...)

plusieurs solutions :

- stockage contigu (CD-ROM)
- liste chaînée de blocs
- liste chaînée de numéros de blocs (*File Allocation Table*)

Où et comment stocker les fichiers ?

un fichier, c'est...

- du contenu
- des attributs ou méta-données (type, taille, permissions, dates...)

plusieurs solutions :

- stockage contigu (CD-ROM)
- liste chaînée de blocs
- liste chaînée de numéros de blocs (*File Allocation Table*)
- table d'i-nœuds, regroupant les attributs et les adresses des blocs (mais pas les noms des fichiers)

Où et comment stocker les fichiers ?

un fichier, c'est...

- du contenu
- des attributs ou méta-données (type, taille, permissions, dates...)

plusieurs solutions :

- stockage contigu (CD-ROM)
- liste chaînée de blocs
- liste chaînée de numéros de blocs (*File Allocation Table*)
- table d'i-nœuds, regroupant les attributs et les adresses des blocs (mais pas les noms des fichiers)
- *Master File Table* de NTFS : suite d'enregistrements décrivant les attributs ou donnant l'adresse des blocs où ils se trouvent (*y compris le nom et les données*)

Où et comment stocker les fichiers ?

un fichier, c'est...

- du contenu
- des attributs ou méta-données (type, taille, permissions, dates...)

plusieurs solutions :

- stockage contigu (CD-ROM)
- liste chaînée de blocs
- liste chaînée de numéros de blocs (*File Allocation Table*)
- table d'i-nœuds, regroupant les attributs et les adresses des blocs (mais pas les noms des fichiers)
- *Master File Table* de NTFS : suite d'enregistrements décrivant les attributs ou donnant l'adresse des blocs où ils se trouvent (*y compris le nom et les données*)
- ...

Consultation des i-nœuds

```
int stat(const char *pathname, struct stat *statbuf);
int lstat(const char *pathname, struct stat *statbuf);
int fstat(int fd, struct stat *statbuf);
int fstatat(int fd, const char *path, struct stat *buf, int flag);
```

remplissent une `struct stat` avec les caractéristiques de l'i-nœud référencé en paramètre et retournent 0, ou -1 en cas d'erreur, précisée par `errno` (`ENOENT` ou `EACCESS` par exemple)

distinction entre `stat()` et `lstat()` : si `pathname` désigne un lien symbolique, alors `lstat()` fournit des informations sur le lien symbolique lui-même, tandis que `stat()` suit la chaîne de liens jusqu'au fichier cible

Consultation des i-nœuds

```
int stat(const char *pathname, struct stat *statbuf);
int lstat(const char *pathname, struct stat *statbuf);
int fstat(int fd, struct stat *statbuf);
int fstatat(int fd, const char *path, struct stat *buf, int flag);
```

remplissent une `struct stat` avec les caractéristiques de l'i-nœud et retournent 0, ou -1 en cas d'erreur, précisée par `errno` (`ENOENT` ou `EACCESS` par exemple)

le type `struct stat` contient (entre autres) les champs suivants :

```
struct stat {
  dev_t     st_dev;       /* ID of device containing file */
  ino_t     st_ino;       /* Inode number */
  mode_t    st_mode;      /* File type and mode */
  uid_t     st_uid;       /* User ID of owner */
  off_t     st_size;      /* Total size, in bytes */
  /* ... */
};
```

Consultation des i-nœuds

Par exemple, pour déterminer un numéro d'i-nœud :

```
struct stat st;  /* déclaration préalable d'une struct stat */
if (lstat("toto", &st)==-1) perror("lstat␣toto");
else printf("inoeud␣numéro␣:␣%ld\n", st.st_ino);
```

Consultation des i-nœuds

Par exemple, pour déterminer un numéro d'i-nœud :

```
struct stat st;  /* déclaration préalable d'une struct stat */
if (lstat("toto", &st)==-1) perror("lstat toto");
else printf("inoeud numéro : %ld\n", st.st_ino);
```

Mais d'autres champs sont plus compliqués à manipuler :

- `st.st_atime`, `st.st_ctime`, `st.st_mtime` sont des `struct timespec`

Consultation des i-nœuds

Par exemple, pour déterminer un numéro d'i-nœud :

```
struct stat st;  /* déclaration préalable d'une struct stat */
if (lstat("toto", &st)==-1) perror("lstat toto");
else printf("inoeud numéro : %ld\n", st.st_ino);
```

Mais d'autres champs sont plus compliqués à manipuler :

- `st.st_atime`, `st.st_ctime`, `st.st_mtime` sont des `struct timespec`
- `st.st_uid` et `st.st_gid` sont les identifiants de l'utilisateur et du groupe propriétaires ; pour déterminer leurs noms, il faut se référer au fichier des mots de passe, par exemple à l'aide de :
  `struct passwd *getpwuid(uid_t uid);`

Consultation des i-nœuds

Par exemple, pour déterminer un numéro d'i-nœud :

```
struct stat st;  /* déclaration préalable d'une struct stat */
if (lstat("toto", &st)==-1) perror("lstat toto");
else printf("inoeud numéro : %ld\n", st.st_ino);
```

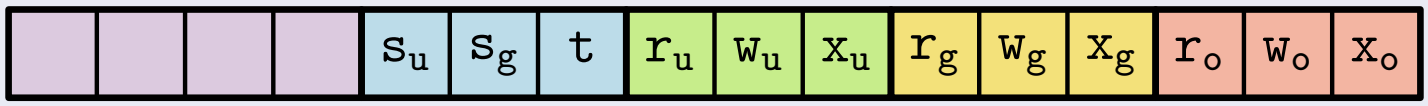
Mais d'autres champs sont plus compliqués à manipuler :

- `st.st_atime`, `st.st_ctime`, `st.st_mtime` sont des `struct timespec`
- `st.st_uid` et `st.st_gid` sont les identifiants de l'utilisateur et du groupe propriétaires ; pour déterminer leurs noms, il faut se référer au fichier des mots de passe, par exemple à l'aide de :
  `struct passwd *getpwuid(uid_t uid);`
- `st.st_mode` est un entier qui agrège deux informations : le type et les droits du fichier

Interprétation du champ st_mode

le champ st_mode est constitué de 2 octets, soit 16 bits :

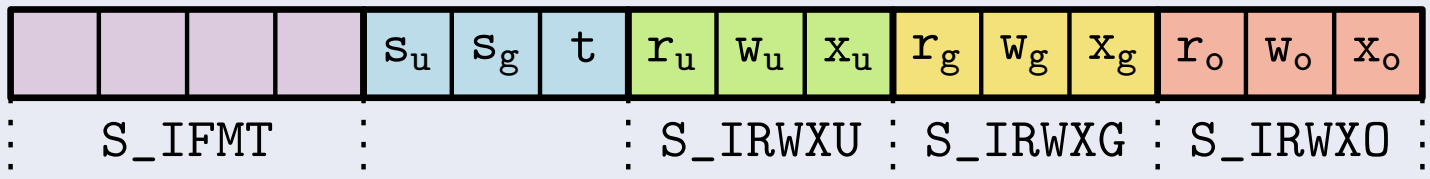
4 pour le type, puis 4 fois 3 pour les droits :



Interprétation du champ st_mode

le champ st_mode est constitué de 2 octets, soit 16 bits :

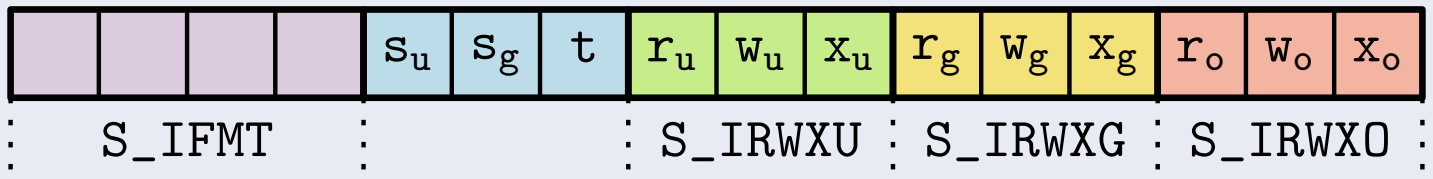
4 pour le type, puis 4 fois 3 pour les droits :



l'interprétation passe par des combinaisons logiques bit à bit avec des masques, comme par exemple S_IFMT=0170000, c'est-à-dire 4 bits 1 suivis de 12 bits 0, ou S_IWOTH=02, c'est-à-dire un unique 1 en avant-dernière position

Interprétation du champ st_mode

le champ st_mode est constitué de 2 octets, soit 16 bits :
4 pour le type, puis 4 fois 3 pour les droits :



l'interprétation passe par des combinaisons logiques bit à bit avec des masques, comme par exemple S_IFMT=0170000, c'est-à-dire 4 bits 1 suivis de 12 bits 0, ou S_IWOTH=02, c'est-à-dire un unique 1 en avant-dernière position

Exemple, pour tester si un fichier est un répertoire :

```
struct stat st;
if (lstat("toto", &st)==-1) perror("stat␣toto");
if ((st.st_mode & S_IFMT) == S_IFDIR) { /* ... */ }
```

ou de manière équivalente :

```
if (S_ISDIR(st.st_mode)) { /* ... */ }
```

Modification des i-nœuds

pour changer les droits :

```
int chmod(const char *path, mode_t mode);
int fchmod(int fd, mode_t mode);
int fchmodat(int fd, const char *path, mode_t mode, int flag);
```

pour changer les propriétaires :

```
int chown(const char *path, uid_t owner, gid_t group);
int fchown(int fd, uid_t owner, gid_t group);
```

pour changer les dates :

```
int utimes(const char *path, const struct timeval times[2]);
int futimes(int fd, const struct timeval times[2]);
```

pour changer la taille :

```
int truncate(const char *path, off_t length);
int ftruncate(int fd, off_t length);
```